

# DevOps Homelab Stack

Guide technique complet

Deploiement automatise de zero a la production

---

|                      |                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Auteur</b>        | Lyes Chaya                                                                                                          |
| <b>GitHub</b>        | <a href="https://github.com/VOTRE_USERNAME/devops-homelab-stack">github.com/VOTRE_USERNAME/devops-homelab-stack</a> |
| <b>Environnement</b> | Ubuntu 24 — PC i7 / 32 Go RAM                                                                                       |
| <b>Date</b>          | 2026                                                                                                                |

# Table des matières

---

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduction au DevOps</b>                                  | <b>3</b>  |
| 1.1       | Definition et contexte . . . . .                               | 3         |
| 1.2       | Architecture de la solution . . . . .                          | 3         |
| 1.3       | Description des composants techniques . . . . .                | 3         |
| <b>2</b>  | <b>Preparation de l'environnement</b>                          | <b>4</b>  |
| 2.1       | Mise a jour du systeme et installation des prerequis . . . . . | 4         |
| <b>3</b>  | <b>Installation de Docker</b>                                  | <b>4</b>  |
| 3.1       | Presentation . . . . .                                         | 4         |
| 3.2       | Procedure d'installation . . . . .                             | 4         |
| 3.3       | Validation de l'installation . . . . .                         | 4         |
| <b>4</b>  | <b>Deploiement du cluster k3s</b>                              | <b>4</b>  |
| 4.1       | Presentation . . . . .                                         | 4         |
| 4.2       | Procedure d'installation . . . . .                             | 5         |
| 4.3       | Verification de l'etat du cluster . . . . .                    | 5         |
| <b>5</b>  | <b>Installation de kubectl et Helm</b>                         | <b>5</b>  |
| 5.1       | kubectl — Interface de controle Kubernetes . . . . .           | 5         |
| 5.2       | Helm — Gestionnaire de packages Kubernetes . . . . .           | 5         |
| <b>6</b>  | <b>Installation et configuration d'Ansible</b>                 | <b>6</b>  |
| 6.1       | Presentation . . . . .                                         | 6         |
| <b>7</b>  | <b>Deploiement d'ArgoCD</b>                                    | <b>6</b>  |
| 7.1       | Presentation et principe GitOps . . . . .                      | 6         |
| 7.2       | Procedure d'installation . . . . .                             | 6         |
| 7.3       | Acces a l'interface de gestion . . . . .                       | 6         |
| <b>8</b>  | <b>Initialisation du depot GitHub</b>                          | <b>7</b>  |
| 8.1       | Creation du depot . . . . .                                    | 7         |
| 8.2       | Configuration locale . . . . .                                 | 7         |
| <b>9</b>  | <b>Developpement et structuration du projet</b>                | <b>8</b>  |
| 9.1       | Application Flask . . . . .                                    | 8         |
| 9.2       | Dockerfile — Specification de l'image conteneur . . . . .      | 9         |
| 9.3       | Ansible — Playbook de provisionnement . . . . .                | 9         |
| 9.4       | Helm Chart — Packaging Kubernetes . . . . .                    | 10        |
| 9.5       | ArgoCD Application — Manifeste GitOps . . . . .                | 11        |
| 9.6       | GitHub Actions — Pipeline CI/CD . . . . .                      | 12        |
| <b>10</b> | <b>Mise en place de la supervision</b>                         | <b>14</b> |
| 10.1      | Presentation de la stack de monitoring . . . . .               | 14        |
| 10.2      | Deploiement via Helm . . . . .                                 | 14        |
| 10.3      | Acces aux interfaces de supervision . . . . .                  | 14        |
| 10.4      | Dashboards Grafana recommandes . . . . .                       | 14        |

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| <b>11</b> | <b>Referentiel des commandes operationnelles</b> | <b>15</b> |
| <b>12</b> | <b>Glossaire technique</b>                       | <b>15</b> |

# Introduction

---

## Presentation du DevOps

Le DevOps est une approche visant a unifier les equipes de developpement logiciel et d'exploitation infrastructure. L'objectif est de reduire le cycle de vie du developpement tout en garantissant la fiabilite des livraisons en production.

### Benefices cles :

- Reduction du temps de mise en production
- Automatisation des tests et de la validation
- Traçabilité complete des deployments via Git
- Elimination des erreurs humaines

## Architecture de la solution

Le flux de deploiement automatise suit le schema suivant :

`git push` → **GitHub Actions** → **Docker** → **Registry** → **ArgoCD** → **k3s** → **Grafana**

## Description des composants

| Composant             | Categorie              | Role                           |
|-----------------------|------------------------|--------------------------------|
| <b>Docker</b>         | Conteneurisation       | Encapsulation de l'application |
| <b>Ansible</b>        | Infrastructure as Code | Provisionnement des serveurs   |
| <b>k3s</b>            | Orchestration          | Gestion des conteneurs         |
| <b>Helm</b>           | Packaging              | Deploiement Kubernetes         |
| <b>GitHub Actions</b> | CI/CD                  | Pipeline automatise            |
| <b>ArgoCD</b>         | GitOps                 | Synchronisation Git/cluster    |
| <b>Prometheus</b>     | Observabilite          | Collecte des metriques         |
| <b>Grafana</b>        | Visualisation          | Tableaux de bord               |

## Preparation de l'environnement

---

### Mise a jour du systeme

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y curl git wget vim
```

### Note sur le terminal Ubuntu

Dans le terminal Ubuntu, les raccourcis de copier-coller sont **Ctrl+Shift+C** pour copier et **Ctrl+Shift+V** pour coller. Dans VS Code, **Ctrl+C** et **Ctrl+V** fonctionnent normalement.

## Installation de Docker

---

## Presentation

Docker permet d'encapsuler une application et ses dependances dans un conteneur portable. Ce conteneur fonctionne de maniere identique sur tout environnement, eliminant les problemes de compatibilite entre developpement et production.

## Installation

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker
docker --version
```

## Validation

```
docker run hello-world
```

Resultat attendu : Hello from Docker!

## Deploiement du cluster k3s

---

### Presentation

k3s est une distribution Kubernetes certifiee optimisee pour les environnements a ressources contraintes. Il ne requiert que 512 Mo de RAM et est ideal pour un usage local.

### Installation

```
curl -sfL https://get.k3s.io | sh -
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
echo 'export KUBECONFIG=~/.kube/config' >> ~/.bashrc
source ~/.bashrc
```

### Validation

```
kubectl get nodes
```

Resultat attendu : le noeud doit afficher le statut Ready.

## Installation de kubectl et Helm

---

### kubectl

kubectl est l'outil en ligne de commande pour interagir avec le cluster Kubernetes.

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.  
key \  
| sudo gpg --dearmor \  
-o /etc/apt/keyrings/kubernetes-apt-keyring.gpg  
  
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.  
gpg] \  
https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' \  
| sudo tee /etc/apt/sources.list.d/kubernetes.list  
  
sudo apt update && sudo apt install -y kubectl
```

## Helm

Helm est le gestionnaire de packages Kubernetes. Il permet de deployer des applications via des Charts paramétrables.

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/  
get-helm-3 | bash  
helm version
```

## Installation d'Ansible

---

### Presentation

Ansible est un outil d'Infrastructure as Code permettant de provisionner et configurer des serveurs via des fichiers YAML versionnés appelés playbooks.

```
sudo apt install -y ansible  
ansible --version  
ansible localhost -m ping
```

Resultat attendu : localhost | SUCCESS => { "ping": "pong" }

## Deploiement d'ArgoCD

---

### Presentation

ArgoCD est un opérateur Kubernetes implementant le paradigme GitOps. Il synchronise en continu l'état déclaré dans Git avec l'état effectif du cluster.

*Principe fondamental : l'état du dépôt Git est la source de vérité unique. Tout écart est automatiquement corrigé.*

### Installation

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f \  
  https://raw.githubusercontent.com/argoproj/argo-cd/stable/  
  manifests/install.yaml  
kubectl get pods -n argocd -w
```

## Acces a l'interface

```
kubectl port-forward svc/argocd-server -n argocd 8888:443
```

Ouvrir : <https://localhost:8888>

```
# Recuperer le mot de passe administrateur  
kubectl -n argocd get secret argocd-initial-admin-secret \  
  -o jsonpath="{.data.password}" | base64 -d
```

Identifiant : admin / Mot de passe : resultat de la commande.

# Initialisation du depot GitHub

---

## Creation du depot

1. Se connecter sur `github.com`
2. Cliquer sur **New repository**
3. Nom : `devops-homelab-stack`
4. Description : Full DevOps pipeline: Docker + k3s + Helm + GitHub Actions + ArgoCD
5. Visibilite : **Public**
6. Cliquer **Create repository**

## Configuration locale

```
git clone https://github.com/VOTRE_USERNAME/devops-homelab-stack
.git
cd devops-homelab-stack
git config --global user.email "votre@email.com"
git config --global user.name "VotreNom"
git config --global pull.rebase true
```

## Securite — Fichiers sensibles

---

**Regle fondamentale** : les informations sensibles (tokens, mots de passe, adresses IP, clefs SSH) ne doivent jamais etre versionnees dans Git.

## Fichier `.gitignore`

Creer a la racine du projet :

```
.env
*.env
secrets.yaml
*secrets*
*token*
*.pem
*.key
*.crt
kubeconfig
```

## Variables sensibles dans GitHub Actions

Les tokens et identifiants sont stockes dans **GitHub** → **Settings** → **Secrets and variables** → **Actions**.

Dans le pipeline, on les reference sans jamais ecrire leur valeur :



```
# Correct : reference a un secret stocke
run: echo "${{ secrets.MON_TOKEN }}" | docker login

# Incorrect : valeur ecrite directement dans le code
run: echo "ghp_XXXXXXXXXXXXXXXXXX" | docker login
```

## Developpement du projet

---

### Application Flask — app/main.py

```
mkdir app
```

```
from flask import Flask, jsonify
from prometheus_flask_exporter import PrometheusMetrics
import os

app = Flask(__name__)
metrics = PrometheusMetrics(app)

@app.route("/")
def home():
    return jsonify({
        "message": "DevOps Homelab Stack",
        "version": os.getenv("APP_VERSION", "1.0.0"),
        "env": os.getenv("APP_ENV", "dev")
    })

@app.route("/health")
def health():
    return jsonify({"status": "ok"}), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Fichier app/requirements.txt :

```
flask==3.0.0
gunicorn==21.2.0
prometheus-flask-exporter==0.23.1
```

```
git add app/
git commit -m "feat: add Flask application"
git push origin main
```

### Dockerfile

```
FROM python:3.11-slim
```

```
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ .
ENV APP_ENV=production
ENV APP_VERSION=1.0.0
EXPOSE 5000
CMD ["python", "main.py"]
```

```
git add Dockerfile
git commit -m "feat: add Dockerfile"
git push origin main
```

## Ansible — Provisionnement

Fichier `ansible/inventory/hosts.ini` :

```
[devops_server]
localhost ansible_connection=local ansible_user=VOTRE_USER
```

Fichier `ansible/playbooks/setup-k3s.yml` :

```
---
- name: Provisionnement du serveur DevOps
  hosts: devops_server
  become: true
  tasks:
    - name: Installation des dependances
      apt:
        name: [curl, git, docker.io]
        state: present
        update_cache: yes
    - name: Deploiement de k3s
      shell: curl -sL https://get.k3s.io | sh -
    - name: Configuration du kubeconfig
      copy:
        src: /etc/rancher/k3s/k3s.yaml
        dest: "/home/VOTRE_USER/.kube/config"
        remote_src: yes
        owner: VOTRE_USER
        mode: '0600'
```

```
git add ansible/
git commit -m "feat: add Ansible provisioning playbook"
git push origin main
```

## Helm Chart — Packaging Kubernetes

Fichier `helm/myapp/Chart.yaml` :

```
apiVersion: v2
name: myapp
description: Application Flask sur k3s
type: application
version: 0.1.0
appVersion: "1.0.0"
```

Fichier helm/myapp/values.yaml :

```
# Le champ tag est mis a jour automatiquement par le pipeline CI
/CD
replicaCount: 1
image:
  repository: ghcr.io/VOTRE_USERNAME/devops-homelab-stack
  tag: "latest"
  pullPolicy: IfNotPresent
service:
  type: ClusterIP
  port: 80
  targetPort: 5000
```

Fichier helm/myapp/templates/deployment.yaml :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-app
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}
    spec:
      containers:
        - name: app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.targetPort }}
```

Fichier helm/myapp/templates/service.yaml :

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-svc
```

```
spec:
  selector:
    app: {{ .Release.Name }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}
    type: {{ .Values.service.type }}
```

```
git add helm/
git commit -m "feat: add Helm chart"
git push origin main
```

## ArgoCD — Manifeste GitOps

Fichier argocd/application.yaml :

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/VOTRE_USERNAME/devops-homelab-stack.git
    targetRevision: main
    path: helm/myapp
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

```
kubectl apply -f argocd/application.yaml
git add argocd/
git commit -m "feat: add ArgoCD application manifest"
git push origin main
```

## GitHub Actions — Pipeline CI/CD

Fichier .github/workflows/ci.yml :

```
name: CI/CD Pipeline
on:
```

```

push:
  branches: [main]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout du code source
        uses: actions/checkout@v3

      - name: Authentification au Container Registry
        run: |
          echo "${{ secrets.GITHUB_TOKEN }}" | \
          docker login ghcr.io -u "${{ github.actor }}" --password
            -stdin

      - name: Construction de l'image Docker
        run: |
          IMAGE="ghcr.io/${{ github.repository }}"
          TAG="${{ github.sha }}"
          docker build -t ${IMAGE}:${TAG} .
          docker tag ${IMAGE}:${TAG} ${IMAGE}:latest

      - name: Validation fonctionnelle
        run: |
          IMAGE="ghcr.io/${{ github.repository }}"
          TAG="${{ github.sha }}"
          docker run -d --name test-app -p 5000:5000 ${IMAGE}:${TAG}
          sleep 5
          curl -f http://localhost:5000/health
          docker stop test-app

      - name: Publication de l'image
        run: |
          IMAGE="ghcr.io/${{ github.repository }}"
          TAG="${{ github.sha }}"
          docker push ${IMAGE}:${TAG}
          docker push ${IMAGE}:latest

      - name: Mise a jour du tag Helm
        run: |
          TAG="${{ github.sha }}"
          sed -i "s|tag:.*|tag: \"${TAG}\"|g" helm/myapp/values.
            yaml
          git config user.email "github-actions@github.com"
          git config user.name "GitHub Actions"
          git add helm/myapp/values.yaml
          git commit -m "ci: update image tag [skip ci]"
          git push

```

**Note :** Les variables `github.sha`, `github.repository` et `secrets.GITHUB_TOKEN` sont injectees automatiquement par GitHub. Aucune valeur sensible n'est ecrite dans le code.

```
git add .github/
git commit -m "ci: add GitHub Actions pipeline"
git push origin main
```

## Supervision avec Prometheus et Grafana

---

### Installation

```
helm repo add prometheus-community \
  https://prometheus-community.github.io/helm-charts
helm repo update
helm install monitoring prometheus-community/kube-prometheus-
  stack \
  --namespace monitoring --create-namespace
kubectl get pods -n monitoring -w
```

### Acces a Grafana

```
export POD_NAME=$(kubectl --namespace monitoring get pod \
  -l "app.kubernetes.io/name=grafana" -oname)
kubectl --namespace monitoring port-forward $POD_NAME 3000
```

Ouvrir : `http://localhost:3000`

Identifiant : admin / Mot de passe : prom-operator

### Acces a Prometheus

```
kubectl port-forward svc/prometheus-operated -n monitoring
  9090:9090
```

Ouvrir : `http://localhost:9090`

## Referentiel des commandes

---

| Commande                                        | Description                 |
|-------------------------------------------------|-----------------------------|
| <code>kubectl get pods -A</code>                | Inventaire de tous les pods |
| <code>kubectl get nodes</code>                  | Statut du cluster           |
| <code>kubectl top nodes</code>                  | Consommation CPU/RAM        |
| <code>kubectl top pods -A</code>                | Consommation par pod        |
| <code>kubectl logs -n production ...</code>     | Journaux applicatifs        |
| <code>kubectl get applications -n argocd</code> | Applications ArgoCD         |
| <code>helm list -A</code>                       | Releases Helm installes     |
| <code>kubectl get events -A</code>              | Journal des evenements      |
| <code>git pull -rebase origin main</code>       | Synchronisation depot       |

## Glossaire

---

**Conteneur** Instance d'exécution isolée d'une image Docker.

**GitOps** Paradigme où Git est la source de vérité unique pour la gestion de l'infrastructure.

**Helm Chart** Package de ressources Kubernetes paramétrables via un fichier `values.yaml`.

**IaC** Infrastructure as Code : gestion de l'infrastructure via des fichiers de configuration versionnés.

**Namespace** Partitionnement logique d'un cluster Kubernetes.

**Pipeline CI/CD** Sequence automatisée déclenchée par un `git push`.

**Pod** Unité atomique de déploiement Kubernetes.

**Registry** Depot de stockage d'images Docker (ex : `ghcr.io`).

**SHA de commit** Identifiant unique d'un commit Git, utilisé pour tagger les images Docker.

**Sync ArgoCD** Reconciliation entre l'état Git et le cluster.